

CSS

Các khái niệm cơ bản

CSS là gì?

- CSS (Cascading Style Sheet): Là ngôn ngữ được sử dụng để quy định cách thức trình bày của các thẻ HTML trên trang web.



HTML



CSS



Lợi ích của việc sử dụng CSS

- Tiết kiệm thời gian
- Linh hoạt trong thiết kế giao diện
- Tăng tốc độ tải trang web
- Dễ dàng bảo trì các trang web
- Thiết kế trang web tương thích nhiều thiết bị

Cú pháp của CSS

- Cú pháp của CSS được tạo nên bởi 3 thành phần:
 - **Bộ chọn (selector)**: dùng để xác định thẻ HTML được áp dụng qui tắc định dạng.
 - **Thuộc tính (property)**
 - **Giá trị thuộc tính (value)**

Bộ chọn
(selector)

`div` {

Thuộc tính
(property)

`display: block;`
`color: green;`
`background-color: #f5f5f5;`
`border-bottom: 1px solid #cccccc;`

}

Giá trị thuộc tính
(value)

Cách sử dụng CSS

- **Inline CSS:** Viết trực tiếp trong thuộc tính style của thẻ HTML

```
<p style="font-family: Arial; color:#ff0000">  
    Hello The World!  
</p>
```

Cách sử dụng CSS

- **Internal CSS:** Viết trong cặp thẻ `<style>` ở phần `<head>` của trang web.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>CSS Example </title>
    <style>
      p {
        font-family: Arial;
        color: ■ #ff0000;
      }
    </style>
  </head>
  <body>
    <p>Hello The World</p>
  </body>
</html>
```

Cách sử dụng CSS

- **External CSS:** Viết trong file có phần mở rộng là css và liên kết vào file HTML bằng thẻ <link>

```
style.css
p {
    font-family: Arial;
    color: ■ #ff0000;
}
```

```
index.html
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>CSS Example </title>
    <link rel="stylesheet" href="style.css" />
  </head>
  <body>
    <p>Hello The World</p>
  </body>
</html>
```



Thứ tự ưu tiên của CSS

- Inline CSS có độ ưu tiên cao nhất
- Internal CSS và External CSS có độ ưu tiên ngang nhau, cái nào viết sau được ưu tiên hơn

CSS Selectors

- CSS Selector (bộ chọn CSS) được sử dụng để xác định phần tử HTML cần áp dụng các quy tắc CSS.

Xác định phần tử HTML theo tên thẻ:

```
p {  
    color: blue;  
}
```

Xác định phần tử HTML theo id:

```
#header {  
    color: blue;  
}
```

Xác định phần tử HTML theo class:

```
.content {  
    color: blue;  
}
```

Tất cả các thẻ:

```
* {  
    color: blue;  
}
```

Lưu ý: Thứ tự ưu tiên (độ mạnh, yếu) của selector
id → class → thẻ → *

CSS Selector

- Các qui tắc CSS giống nhau có thể nhóm lại thành một nhóm, sử dụng dấu phẩy để phân cách các CSS selector

```
p {  
  color: blue;  
  font-size: 13px;  
}  
#title {  
  color: blue;  
  font-size: 13px;  
}  
.content {  
  color: blue;  
  font-size: 13px;  
}
```



```
p, #title, .content {  
  color: blue;  
  font-size: 13px;  
}
```

CSS Selectors

- Xác định phần tử theo giá trị của thuộc tính, sử dụng cú pháp:

`selector[property="value"]`

Ví dụ:

```
input[type="text"]{  
    font-size: 20px;  
    color: ■ blue;  
}  
  
a[target="_blank"] {  
    color: ■ red;  
    text-decoration: none;  
}
```

CSS Combinators

- CSS Combinator là cách viết kết hợp các CSS selector với nhau, được sử dụng để xác định mối quan hệ giữa các phần tử HTML trong các selector.
- Các combinator cho phép xác định các phần tử HTML dựa trên mối quan hệ không gian, vị trí, hoặc các điều kiện khác giữa chúng.

CSS Combinators

Ví dụ	Đối tượng chịu tác động
<code>div p</code>	Các thẻ p bên trong thẻ div (ở bất kỳ cấp độ nào)
<code>div > p</code>	Các thẻ p là con của thẻ div
<code>div + p</code>	Thẻ p được đặt kế và ngay sau thẻ div
<code>div ~ p</code>	Các thẻ p đồng cấp nằm sau thẻ div (p là “em” của thẻ div)

CSS Compound selector

- Compound selector (bộ chọn kết hợp) là một chuỗi các selector đơn giản không được phân cách bởi các combinator.
- Bộ chọn kết hợp biểu thị một tập hợp các điều kiện đồng thời trên một phần tử HTML.

Ví dụ	Đối tượng chịu tác động
<code>div.content</code>	Các thẻ div có class là content

CSS Pseudo-classes

- Pseudo-classes (lớp giả định) được sử dụng để xác định phần tử HTML dựa trên trạng thái hoặc điều kiện cụ thể nào đó.

Ví dụ	Giải thích
<code>a:hover</code>	
<code>a:link</code>	
<code>a:active</code>	
<code>a:visited</code>	
<code>input:focus</code>	
<code>input:checked</code>	
<code>input:disabled</code>	
<code>input:enabled</code>	

CSS Pseudo-classes

Ví dụ	Giải thích
<code>input:in-range</code>	
<code>input:invalid</code>	
<code>input:optional</code>	
<code>input:out-of-range</code>	
<code>input:read-only</code>	
<code>input:read-write</code>	
<code>input:required</code>	
<code>input:valid</code>	

CSS Pseudo-classes

Ví dụ	Giải thích
<code>tr:first-child</code>	
<code>tr:last-child</code>	
<code>tr:nth-child(2)</code>	
<code>tr:nth-child(odd)</code>	
<code>tr:nth-child(event)</code>	
<code>tr:nth-child(3n+0)</code>	

CSS Pseudo-elements

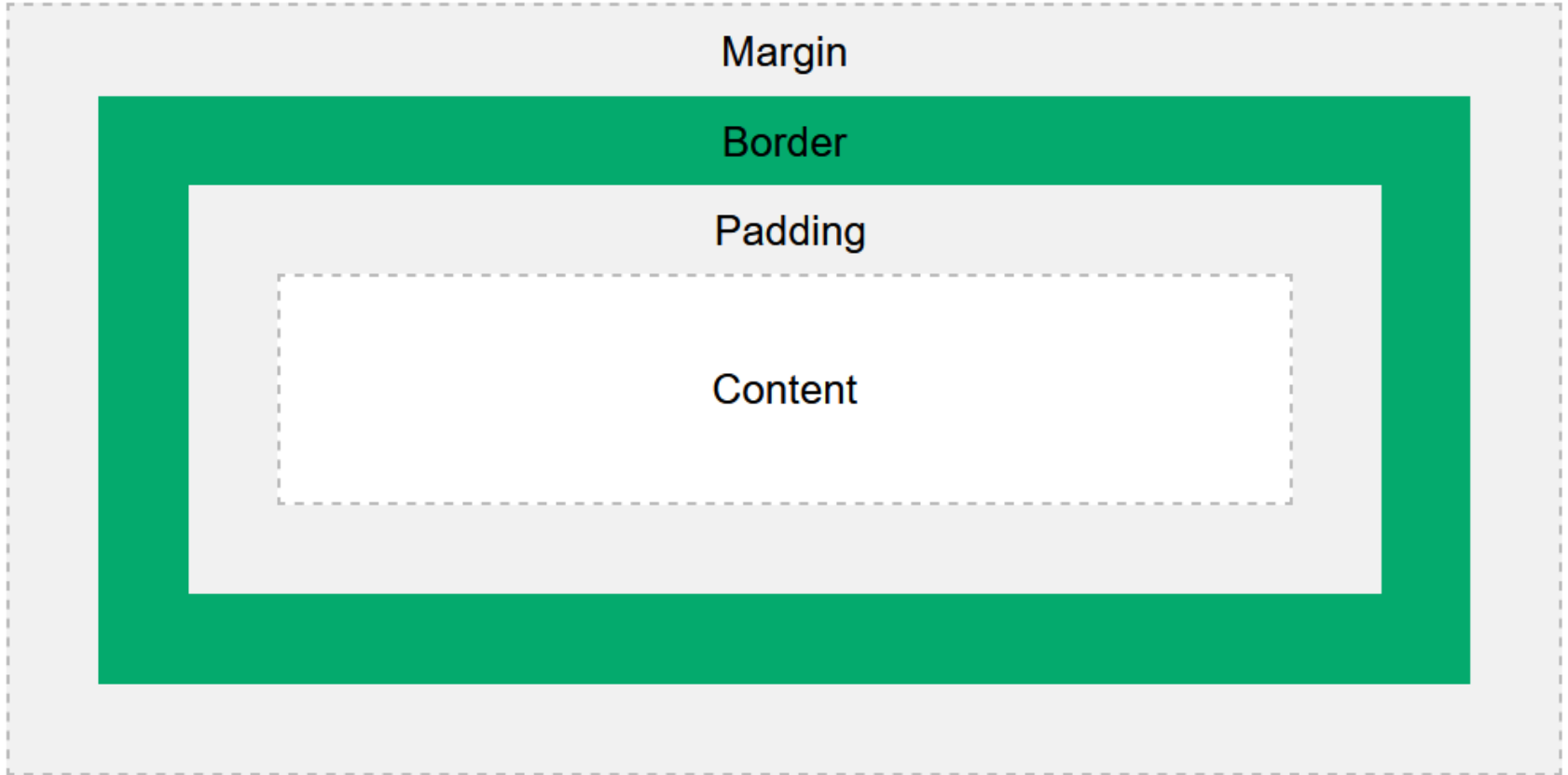
- CSS Pseudo-element (phần tử giả) được sử dụng để tạo style cho các phần được chỉ định của phần tử HTML.

Ví dụ	Giải thích
<code>p::after</code>	
<code>p::before</code>	
<code>p::first-letter</code>	
<code>p::first-line</code>	
<code>::marker</code>	
<code>p::selection</code>	

CSS Box Model

- CSS Box Model là khái niệm mô tả cách mỗi phần tử HTML được hiển thị và tương tác với các phần tử khác trên trang web.
- Mỗi phần tử HTML khi hiển thị trên trình duyệt có thể xem là nằm bên trong một vùng hình chữ nhật bao quanh nó (nằm trong một “hộp”). Bao gồm:
 - **Content**: nội dung của phần tử
 - **Padding**: Vùng đệm giữa đường viền và nội dung của phần tử
 - **Border**: Đường viền bao quanh phần tử
 - **Margin**: Khoảng cách giữa đường viền của phần tử với các phần tử khác lân cận

CSS Box Model



CSS Box Model

- Các thuộc tính liên quan đến CSS Box Model

Thuộc tính	Mô tả
width	Độ rộng của phần tử
height	Chiều cao của phần tử
box-sizing	Xác định cách tính tổng kích thước của phần tử. Thuộc tính này có thể nhận một trong 2 giá trị: • context-box (mặc định) • border-box <pre>* { box-sizing: border-box; }</pre>

CSS Box Model

- Các thuộc tính liên quan đến CSS Box Model

Thuộc tính	Mô tả
margin	Khoảng cách lề của phần tử <code>margin: 10px 20px 30px 40px;</code>
margin-top	Khoảng cách lề bên trên <code>margin-top: 10px;</code>
margin-right	Khoảng cách lề bên phải <code>margin-right: 20px;</code>
margin-bottom	Khoảng cách lề bên dưới <code>margin-bottom: 30px;</code>
margin-left	Khoảng cách lề bên trái <code>margin-left: 40px;</code>

CSS Box Model

- Các thuộc tính liên quan đến CSS Box Model

Thuộc tính	Mô tả
border	Kích thước đường viền bao quanh phần tử <code>border: 1px solid blue;</code>
border-top	Kích thước đường viền bên trên <code>border-top: 1px dotted blue;</code>
border-right	Kích thước đường viền bên phải <code>border-right: 1px dashed blue;</code>
border-bottom	Kích thước đường viền bên dưới <code>border-bottom: 1px double blue;</code>
border-left	Kích thước đường viền bên trái <code>border-left: 1px solid blue;</code>

CSS Box Model

- Các thuộc tính liên quan đến CSS Box Model

Thuộc tính	Mô tả
padding	Kích thước vùng đệm giữa nội dung và đường viền <code>padding: 10px 20px 30px 40px;</code>
padding-top	Kích thước vùng đệm bên trên <code>padding-top: 10px;</code>
padding-right	Kích thước vùng đệm bên phải <code>padding-right: 20px;</code>
padding-bottom	Kích thước vùng đệm bên dưới <code>padding-bottom: 30px;</code>
padding-left	Kích thước vùng đệm bên trái <code>padding-left: 40px;</code>

Sử dụng biến trong CSS

- Biến trong CSS giúp tái sử dụng các giá trị, giúp mã nguồn CSS dễ đọc và linh hoạt khi chỉnh sửa.
- Cách khai báo biến
 - Tên biến phải bắt đầu bởi hai dấu gạch ngang: --
 - Giá trị của biến có thể là bất kỳ giá trị CSS nào như: màu sắc, kích thước, font chữ,...
- Với các biến sử dụng chung trong file CSS, cần được khai báo trong khối `root` theo cú pháp

```
:root {
```

```
}
```

Sử dụng biến trong CSS

Ví dụ:

```
/* Khai báo biến */
:root {
    --main-color: #ff0000;
    --font-size: 16px;
}
/* Sử dụng biến */
body {
    color: var(--main-color);
    font-size: var(--font-size);
}
```

Các thuộc tính CSS thông dụng

Định dạng chữ (text) và đoạn văn bản (paragraph)

Thuộc tính	Mô tả
<code>color</code>	Định dạng màu chữ
<code>text-align</code>	Căn lề văn bản theo chiều ngang
<code>text-align-last</code>	Căn lề dòng cuối cùng của đoạn văn bản theo chiều ngang
<code>vertical-align</code>	Căn lề văn bản theo chiều dọc
<code>direction</code>	Hướng văn bản (viết từ trái qua phải, hoặc viết phải qua trái)
<code>text-transform</code>	Thiết lập cách thể hiện văn bản dưới dạng chữ hoa, chữ thường
<code>text-shadow</code>	Thêm bóng mờ vào văn bản

Định dạng chữ (text) và đoạn văn bản (paragraph)

Thuộc tính	Mô tả
<code>text-indent</code>	Khoảng cách thụt lề dòng đầu tiên của đoạn văn bản
<code>letter-spacing</code>	Khoảng cách giữa các ký tự
<code>word-spacing</code>	Khoảng cách giữa các từ
<code>line-height</code>	Khoảng cách giữa các dòng
<code>white-space</code>	Thiết lập cách xử lý đối với các khoảng trắng dư thừa và xuống dòng khi hiển thị

Định dạng chữ (text) và đoạn văn bản (paragraph)

Thuộc tính	Mô tả
<code>text-decoration-line</code>	Loại đường kẻ sử dụng trang trí cho văn bản
<code>text-decoration-color</code>	Màu đường kẻ sử dụng trang trí cho văn bản
<code>text-decoration-style</code>	Kiểu đường kẻ sử dụng trang trí cho văn bản
<code>text-decoration-thickness</code>	Độ dày của đường kẻ sử dụng trang trí cho văn bản
<code>text-decoration</code>	Định dạng đường kẻ cho văn bản

Cú pháp:

```
text-decoration: line [color] [style] [thickness]
```

Ví dụ:

```
text-decoration: underline red dotted 1px
```

Định dạng font chữ

Thuộc tính	Mô tả
<code>font-family</code>	font chữ sử dụng cho văn bản
<code>font-style</code>	font chữ in nghiêng (italic) hay in thường (normal)
<code>font-weight</code>	font chữ in đậm (bold) hay không in đậm (normal)
<code>font-variant</code>	font chữ in hoa cỡ nhỏ (small-caps)
<code>font-size</code>	Kích thước của chữ
<code>font</code>	Định dạng font chữ

Cú pháp:

```
font: [style] [variant] [weight] size[/line-height] family
```

Ví dụ:

```
font: italic small-caps bold 14px/30px Arial;
```

Định dạng nền (background)

Thuộc tính	Mô tả
<code>background-color</code>	Thiết lập màu nền
<code>background-image</code>	Thiết lập ảnh nền
<code>background-repeat</code>	Cách lặp lại của ảnh nền
<code>background-attachment</code>	Ảnh nền cuộn (scroll) hay cố định (fixed)
<code>background-position</code>	Vị trí xuất hiện của ảnh nền
<code>background-size</code>	Kích thước của ảnh nền
<code>background-clip</code>	Phạm vi hiển thị của ảnh nền
<code>background</code>	

Thuộc tính **display**

- Mặc định, mỗi phần tử HTML khi hiển thị trên trang web sử dụng chế độ hiển thị mặc định là **inline** hoặc **block**:
 - **inline**: Các phần tử sẽ hiển thị trên cùng một hàng với các phần tử khác.
 - **block**: Các phần tử sẽ hiển thị trên một hàng riêng biệt
- Thuộc tính **display** được sử dụng để thiết lập cách hiển thị của phần tử HTML trên trang web.

Thuộc tính **display**

- Một số giá trị phổ biến của thuộc tính **display**

Giá trị	Mô tả
block	Hiển thị phần tử như một khối, chiếm toàn bộ chiều rộng của container cha, không nằm cùng các phần tử khác trên cùng một dòng
inline	Hiển thị phần tử nằm trên cùng dòng với các phần tử khác. Lưu ý thuộc tính width và height không có tác dụng đối với phần tử inline
inline-block	Kết hợp các đặc điểm của block và inline. Cho phép thiết lập chiều rộng (width) và chiều cao (height) cho phần tử, đồng thời nó vẫn nằm trên cùng dòng với các phần tử khác.
none	Ẩn phần tử khỏi trang web

Thuộc tính **display**

- Một số giá trị phổ biến của thuộc tính **display**

Giá trị	Mô tả
table	Hiển thị phần tử như một bảng (ngắt dòng trước và sau phần tử)
table-row	Hiển thị phần tử như một hàng trong bảng
table-cell	Hiển thị phần tử như một hàng trong bảng
inline-table	Phần tử có tính chất như bảng nhưng được hiển thị dưới dạng inline (hiển thị như một bảng, nhưng không chuyển xuống dòng mới như một phần tử block thông thường)

Thuộc tính **position**

- Thuộc tính **position** là một trong những thuộc tính quan trọng trong CSS, được sử dụng để thiết lập vị trí hiển thị của phần tử trên trang web.
- Thuộc tính **position** thường được sử dụng kết hợp với các thuộc tính dùng để thiết lập tọa độ vị trí của phần tử là:
 - **top**
 - **right**
 - **bottom**
 - **left**

Thuộc tính **position**

- Các giá trị của thuộc tính **position**

Giá trị	Mô tả
static	Giá trị mặc định của position. Phần tử sẽ được đặt ở vị trí theo thứ tự mặc định của nó trong bố cục. Các thuộc tính top, right, bottom và left không có hiệu lực trong trường hợp này
relative	Phần tử nằm tại vị trí có tọa độ được xác định dựa trên vị trí mặc định của nó. Nếu sử dụng thuộc tính này thì sẽ phải sử dụng kèm theo với các thuộc tính căn chỉnh tọa độ của phần tử (top, right, bottom, left).

Thuộc tính **position**

- Các giá trị thường dùng của thuộc tính **position**

Giá trị	Mô tả
absolute	Định vị trí tuyệt đối cho thành phần dựa trên thành phần bao ngoài gần nhất có thuộc tính position là relative , fixed hoặc absolute (nếu không có thì theo cửa sổ trình duyệt). Nếu sử dụng thuộc tính này thì sẽ phải sử dụng kèm theo với các thuộc tính căn chỉnh tọa độ của phần tử (top , right , bottom , left).
fixed	Phần tử sẽ được đặt tại vị trí cố định dựa trên trình duyệt và không di chuyển khi cuộn trang. Các thuộc tính top , right , bottom , left được sử dụng để thiết lập vị trí của phần tử

Thuộc tính **overflow**

- Thuộc tính **overflow** xác cách xử lý khi một thành phần bao quanh (phần tử chứa) bị tràn nội dung (text).
- Các giá trị của thuộc tính **overflow**:

Giá trị	Mô tả
visible	Hiển thị toàn bộ nội dung, ngay cả khi nó vượt quá kích thước của phần tử chứa.
hidden	Ẩn nội dung vượt quá kích thước của phần tử chứa.
scroll	Hiển thị thanh cuộn để người dùng có thể cuộn qua nội dung vượt quá kích thước của phần tử chứa.
auto	Tự động hiển thị thanh cuộn khi cần thiết